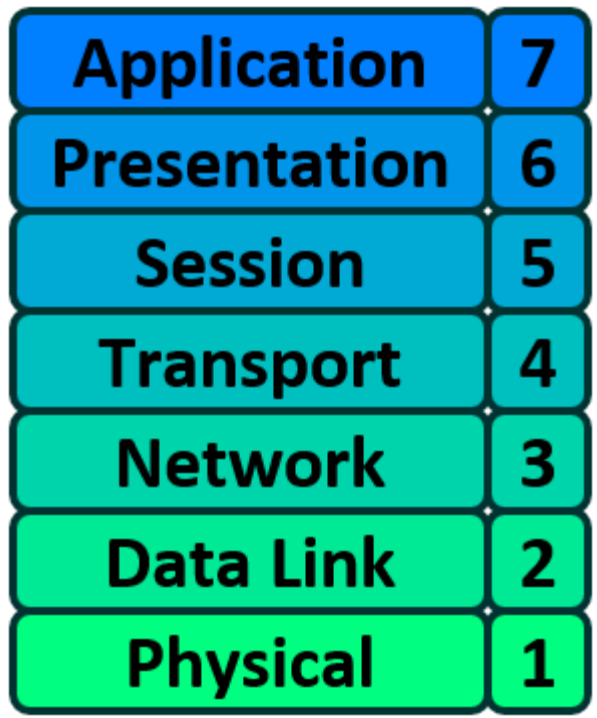
UNIT 3

TRANSPORT LAYER

Overview of Transport Layer in the Internet

- The transport Layer is the fourth layer in the OSI model.
- It is an end-to-end layer used to deliver messages to a host.
- It is termed an end-to-end layer because it provides a point-to-point connection rather than hop-to-hop, between the source host and destination host to deliver the services reliably.
- The unit of data encapsulation in the Transport Layer is a segment**.**


Application
7
Presentation
6
Session
5
Transport
4
Network
3
Data Link
2
Physical
1

Working of Transport Layer

- **The transport layer ensures the reliable transmission of data between systems.**

- **At the sender's side:**

➢ The transport layer receives data (message) from the Application layer and then performs Segmentation, divides the actual message into segments, adds the source and destination's port numbers into the header of the segment, and transfers the message to the Network layer.

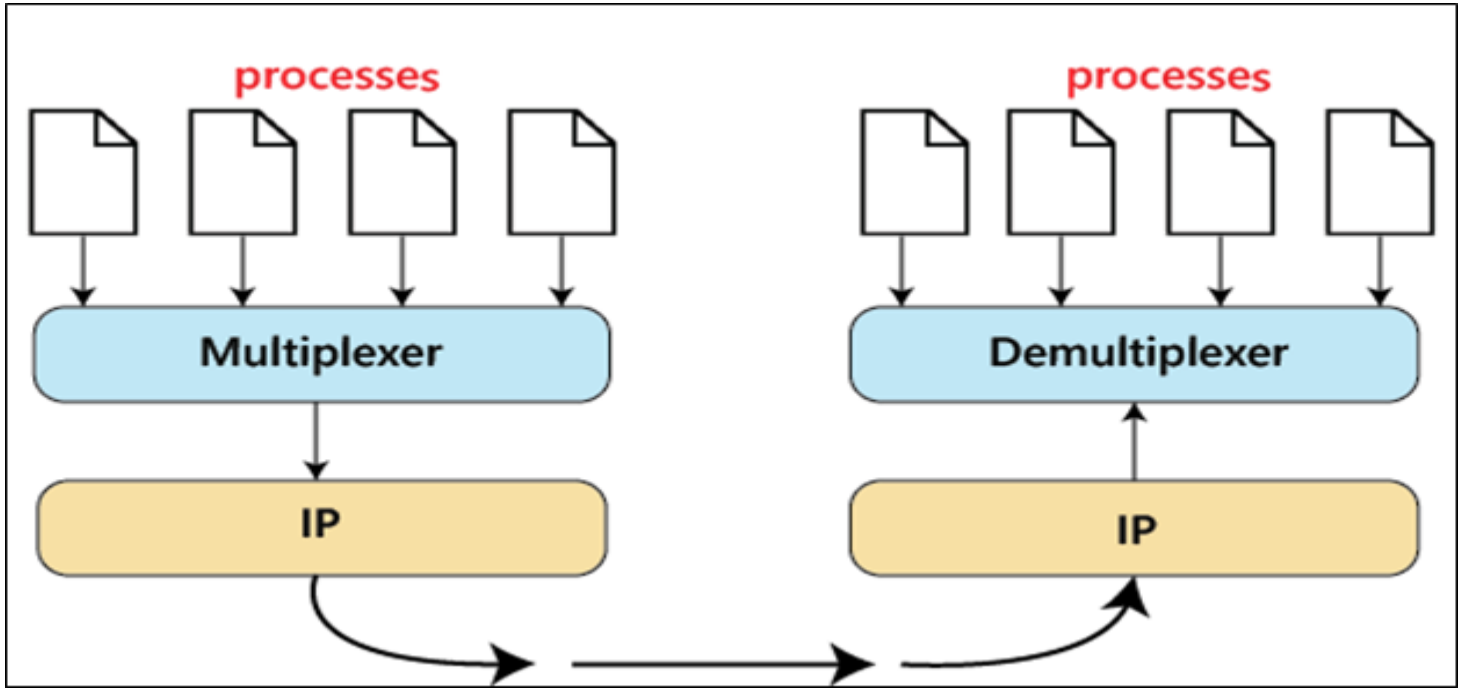
- **At the receiver's side:**

➢ The transport layer receives data from the Network layer, reassembles the segmented data, reads its header, identifies the port number, and forwards the message to the appropriate port in the Application layer

Responsibilities of a Transport Layer

- The Process to Process Delivery
- End-to-End Connection between Hosts
- Multiplexing and Demultiplexing
- Congestion Control
- Data integrity and Error correction
- Flow control

1. The Process to Process Delivery



What is Process-to-Process Delivery?

- When data is sent from one computer to another, it is not enough to deliver it only to the destination computer.
- The data must reach the **correct application/process** running on that computer.

For example:

Computer A

✓Browser

✓Email application

✓YouTube

sends data to

Computer B

✓Web server

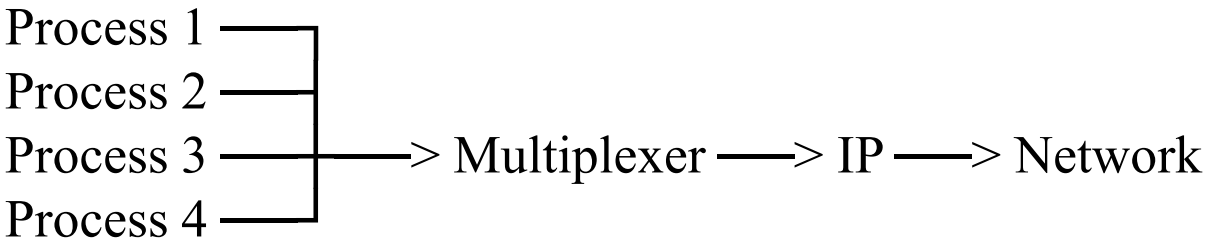
✓Email server

✓Other applications

- The Transport Layer ensures that data reaches the **correct process/application**.

Sender side – Multiplexing

There are several processes sending data:



- The **Multiplexer** collects data from multiple processes.

For example:

•Browser → data
•Email → data
•WhatsApp → data
•YouTube → data

The Transport Layer combines these data streams and sends them toward the Network Layer (**IP**).
This is called **multiplexing**.

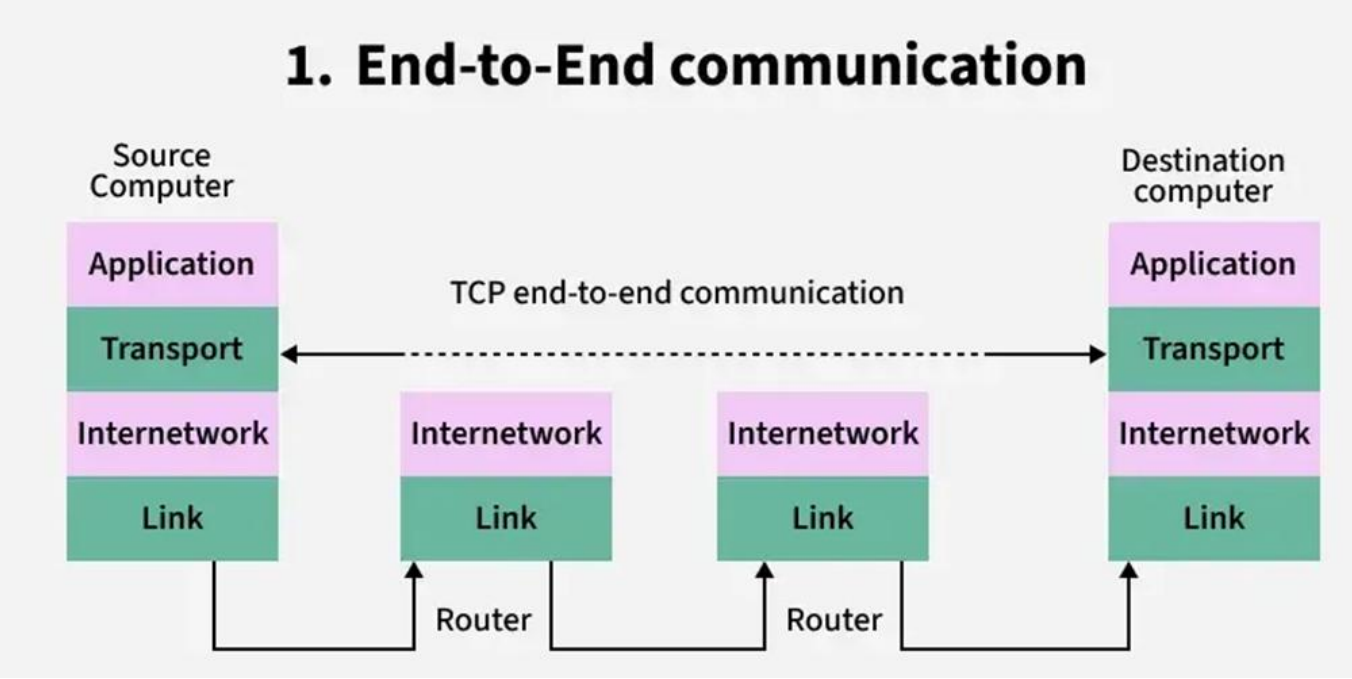
Receiver side – Demultiplexing

- The IP layer receives packets from the network and passes them to the **Demultiplexer**.

Network
↓
IP
↓
Demultiplexer
↓ ↓ ↓ ↓
P1 P2 P3 P4

The **Demultiplexer** examines information in the transport-layer header, especially the **port number**,
and delivers the data to the correct process.
For example:
Port 80/443 → Web application
Port 25 → Email service
Port 53 → DNS service
So, if a packet arrives with the appropriate destination port, the transport layer sends it to the corresponding application.
This process is called **demultiplexing**.

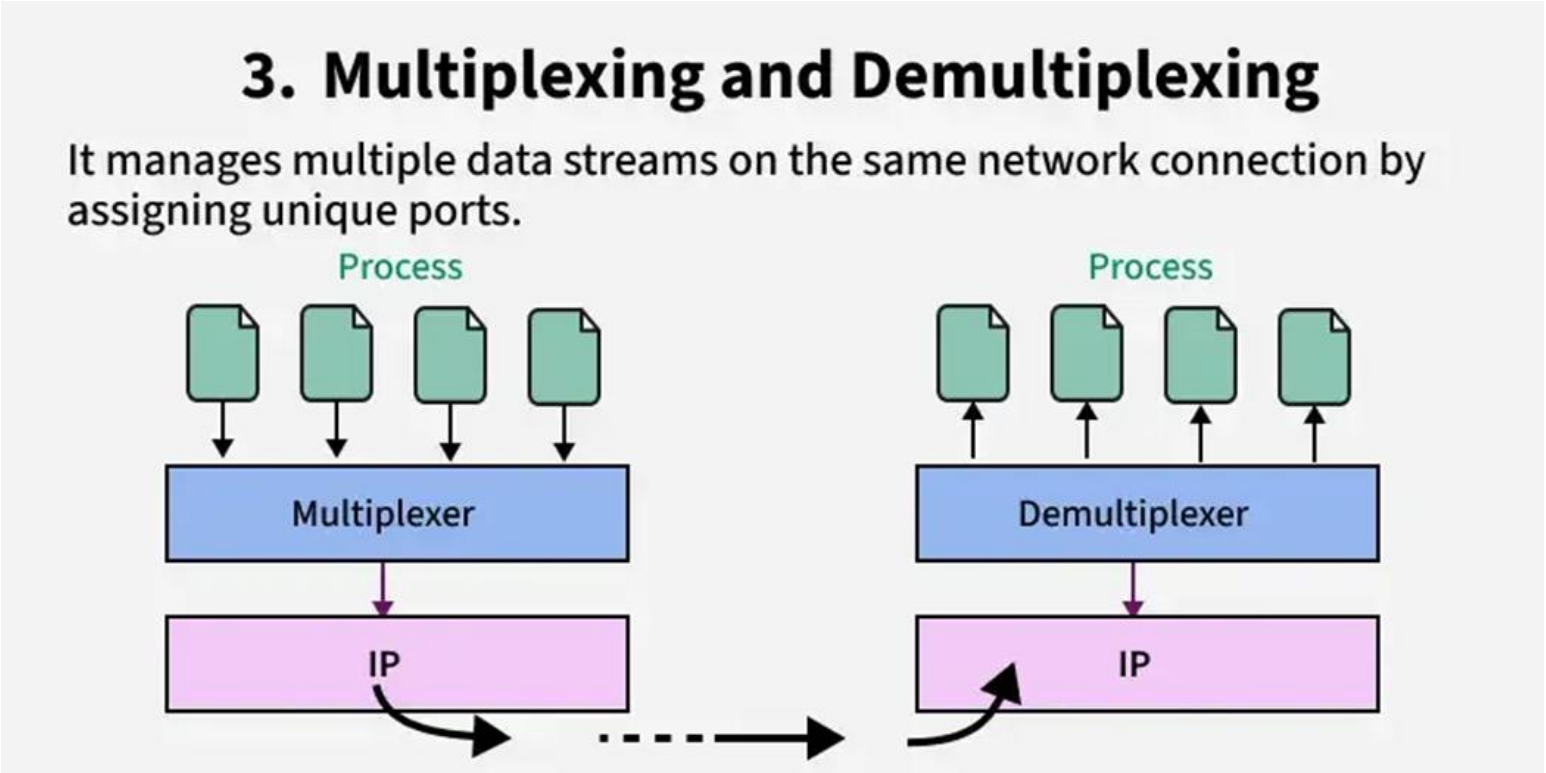
2. End-to-end Connection between Hosts



Continued…

- The transport layer is also responsible for creating the end-to-end connection between hosts for which it mainly uses TCP and UDP.
- TCP is a secure, connection-orientated protocol that uses a handshake protocol to establish a robust connection between two end hosts.
- TCP ensures the reliable delivery of messages and is used in various applications.
- UDP, on the other hand, is a stateless and unreliable protocol that ensures best-effort delivery.
- It is suitable for applications that have little concern with flow or error control and requires sending the bulk of data like video conferencing.

3. Multiplexing and Demultiplexing



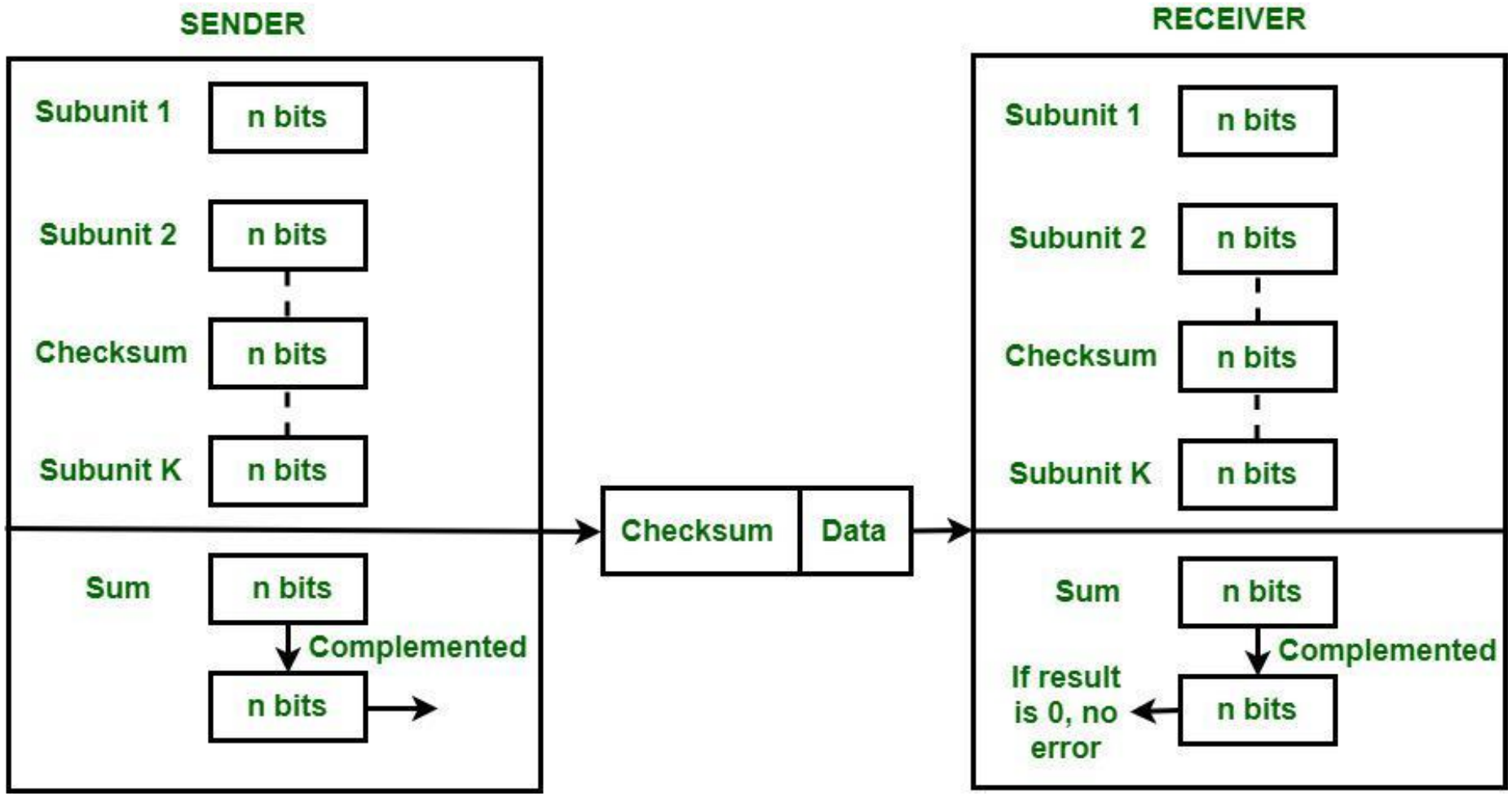
- Multiplexing(many to one) is when data is acquired from several processes from the sender and merged into one packet along with headers and sent as a single packet.
- Multiplexing allows the simultaneous use of different processes over a network that is running on a host.
- The processes are differentiated by their port numbers.
- Similarly, Demultiplexing(one to many) is required at the receiver side when the message is distributed into different processes.
- Transport receives the segments of data from the network layer distributes and delivers it to the appropriate process running on the receiver's machine

4. Congestion Control

- Congestion is a situation in which too many sources over a network attempt to send data and the router buffers start overflowing due to which loss of packets occurs.
- As a result, the retransmission of packets from the sources increases the congestion further.
- In this situation, the Transport layer provides Congestion Control in different ways.
- It uses open-loop congestion control to prevent congestion and closed-loop congestion control to remove the congestion in a network once it occurred.
- TCP provides leaky bucket technique for congestion control.

5. Data integrity and Error Correction

- The transport layer checks for errors in the messages coming from the application layer by using error detection codes, and computing checksums, it checks whether the received data is not corrupted and uses the ACK and NACK services to inform the sender if the data has arrived or not and checks for the integrity of data.


SENDER
Subunit 1
n bits
Subunit 2
n bits
Checksum
n bits
Subunit K
n bits
Sum
n bits
Complemented
n bits
Checksum
Data
RECEIVER
Subunit 1
n bits
Subunit 2
n bits
Checksum
n bits
Subunit K
n bits
Sum
n bits
Complemented
If result is 0, no error
n bits

How Checksums Are Computed

•The sender splits the data into *k* segments of length *n* bits (often 16 bits).

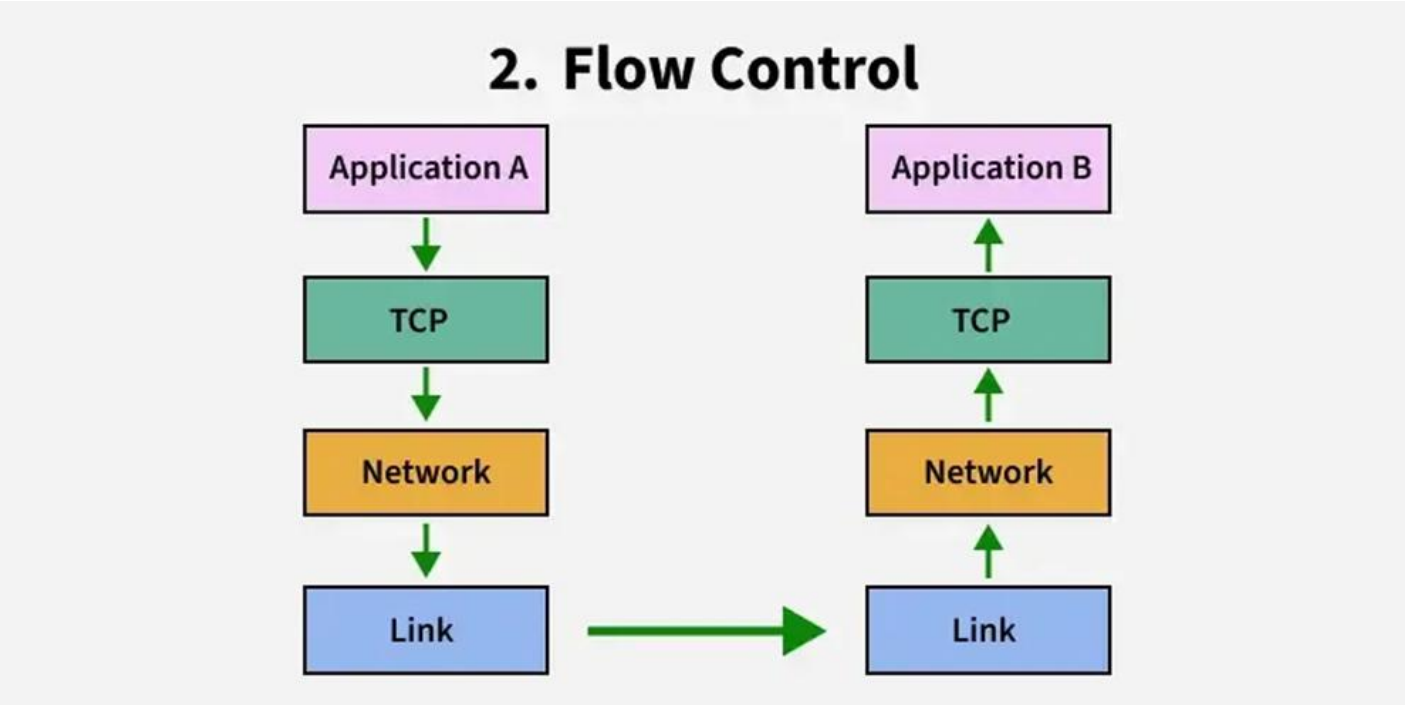
•All segments are summed using one's-complement arithmetic.

•The resulting sum (modulo *n* bits) is inverted (one's complement) — this is the checksum.

•The checksum value is appended to the message before sending

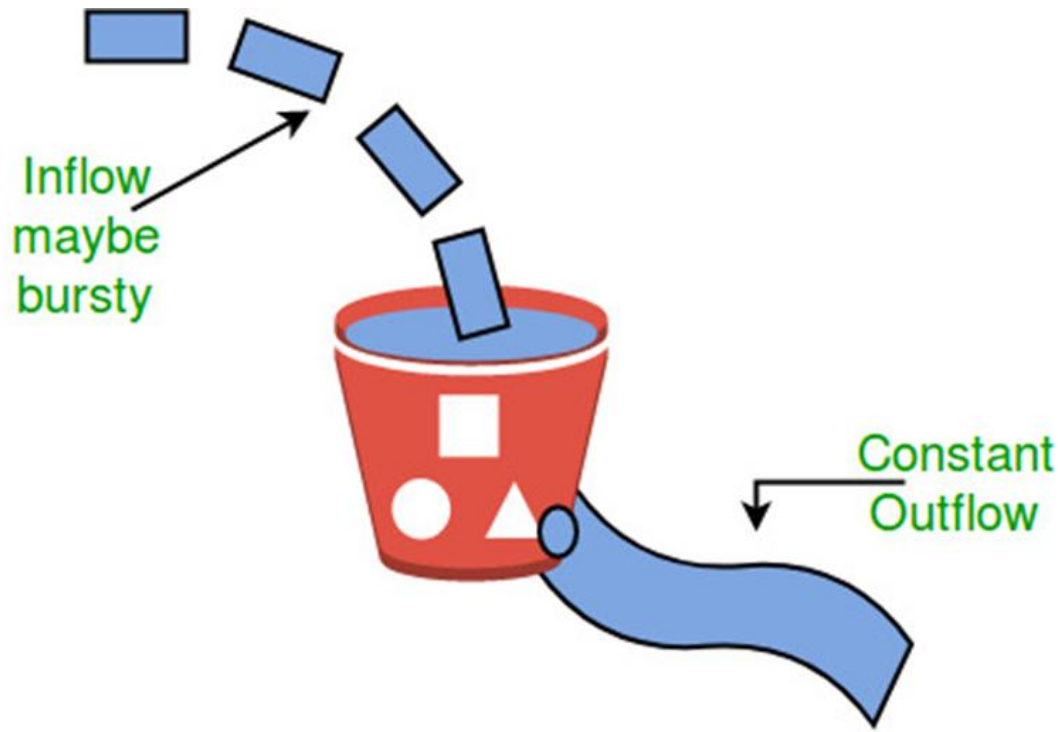
- On the receiver side:
- The receiver adds all segments **plus** the checksum itself, again using one's-complement.
- If the final result equals all zeros, the data is accepted; otherwise, errors are detected and the packet is rejected or re-requested .

6. Flow Control



- The transport layer provides a flow control mechanism between the adjacent layers of the TCP/IP model.
- TCP also prevents data loss due to a fast sender and slow receiver by imposing some flow control techniques.
- It uses the method of sliding window protocol which is accomplished by the receiver by sending a window back to the sender informing the size of data it can receive.

Leaky Bucket Congestion Control Technique



Relation Between Transport Layer And Network Layer

- In the vast realm of computer networks, two critical layers play distinct roles in ensuring data transmission: the Transport Layer and the Network Layer.
- These are essential components of the OSI (Open Systems Interconnection) model, which helps understand how different network functions interact.

Introduction to Network Layer

- Host-to-host delivery is the responsibility of network layer.
- Being the third layer in the OSI model, it handles the logical addressing and routing of data packets across networks.
- It selects the most efficient path for data transmission and forwards packets accordingly

Network Layer Protocols

Internet Protocol (IP):

- The Internet Protocol (IP) is the cornerstone of the network layer.
- It provides logical addressing through IP addresses, enabling devices to identify each other on the internet or any IP-based network.
- IPv4 and IPv6 are the two most widely used versions.

Internet Control Message Protocol (ICMP):

- ICMP works in conjunction with IP to manage error messages and provide network diagnostic functions.
- It allows devices to communicate error conditions, such as unreachable hosts or time-exceeded packets, facilitating efficient troubleshooting.

Routing Information Protocol (RIP):

- RIP is a dynamic routing protocol used in small to medium-sized networks.
- It employs distance-vector algorithms to determine the best path for data packets.

Open Shortest Path First (OSPF):

- OSPF is a popular Interior Gateway Protocol (IGP) used in larger networks.
- It employs a link-state algorithm to determine the shortest path for data packets, ensuring efficient routing and scalability.

Border Gateway Protocol (BGP):

- BGP is an Exterior Gateway Protocol (EGP) designed for inter-domain routing on the internet.
- It enables autonomous systems (AS) to exchange routing information, allowing data to traverse different networks.

Internet Group Management Protocol (IGMP):

- IGMP is essential for managing multicast group memberships.
- It allows devices to join or leave multicast groups, ensuring efficient delivery of multicast traffic within a network.

Address Resolution Protocol (ARP):

- ARP facilitates the translation of IP addresses to MAC (Media Access Control) addresses.
- It plays a crucial role in local area networks (LANs), enabling devices to communicate with each other at the data link layer.

Reverse Address Resolution Protocol (RARP):

- RARP performs the reverse function of ARP by translating MAC addresses into IP addresses.
- It was widely used in older systems but has become less relevant in modern networks.

Internet Group Management Protocol version 3 (IGMPv3):

- IGMPv3 is an enhanced version of IGMP, which allows hosts to send queries specifying the multicast sources they are interested in, optimizing multicast traffic management.

Virtual Router Redundancy Protocol (VRRP):

- VRRP provides high availability in network environments by allowing multiple routers to act as a single virtual router.
- It ensures seamless routing even if one of the routers fails.

Introduction to Transport Layer

- The Transport Layer is responsible for segmenting and reassembling data received from the upper layers into manageable chunks called "segments" before transmitting them to the network layer for further routing and delivery.
- Two prominent transport layer protocols are:

a) Transmission Control Protocol (TCP)

b) User Datagram Protocol (UDP)

Services Provided by the Transport Layer

✓ Reliable Delivery

✓ Error Control

✓ Sequence Control

✓ Flow Control

✓ Congestion Control

✓ Multiplexing

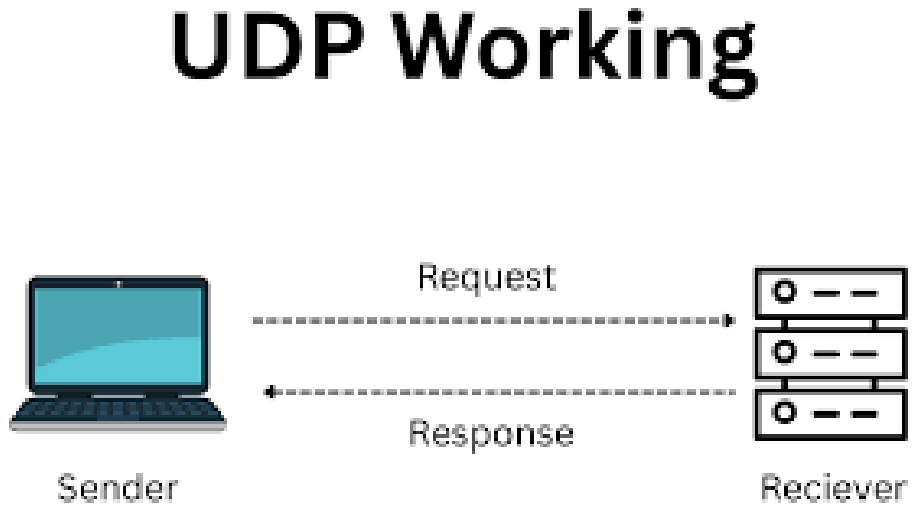
Relation Between Transport and Network Layer

The transport and network layers work hand in hand to ensure smooth data delivery. The transport layer utilizes the services of the network layer to route packets to their destination based on logical addressing.

Aspect	Transport Layer	Network Layer
Primary Function	End-to-end delivery of data packets	Logical addressing and routing of packets
Layer Location in OSI Model	Fourth layer	Third layer
Protocol Examples	TCP, UDP	IP, ICMP
Services Provided	Reliable delivery, error control, flow control	Logical addressing, routing, packet forwarding
Responsibility	Ensures data delivery between applications	Manages data transmission between networks

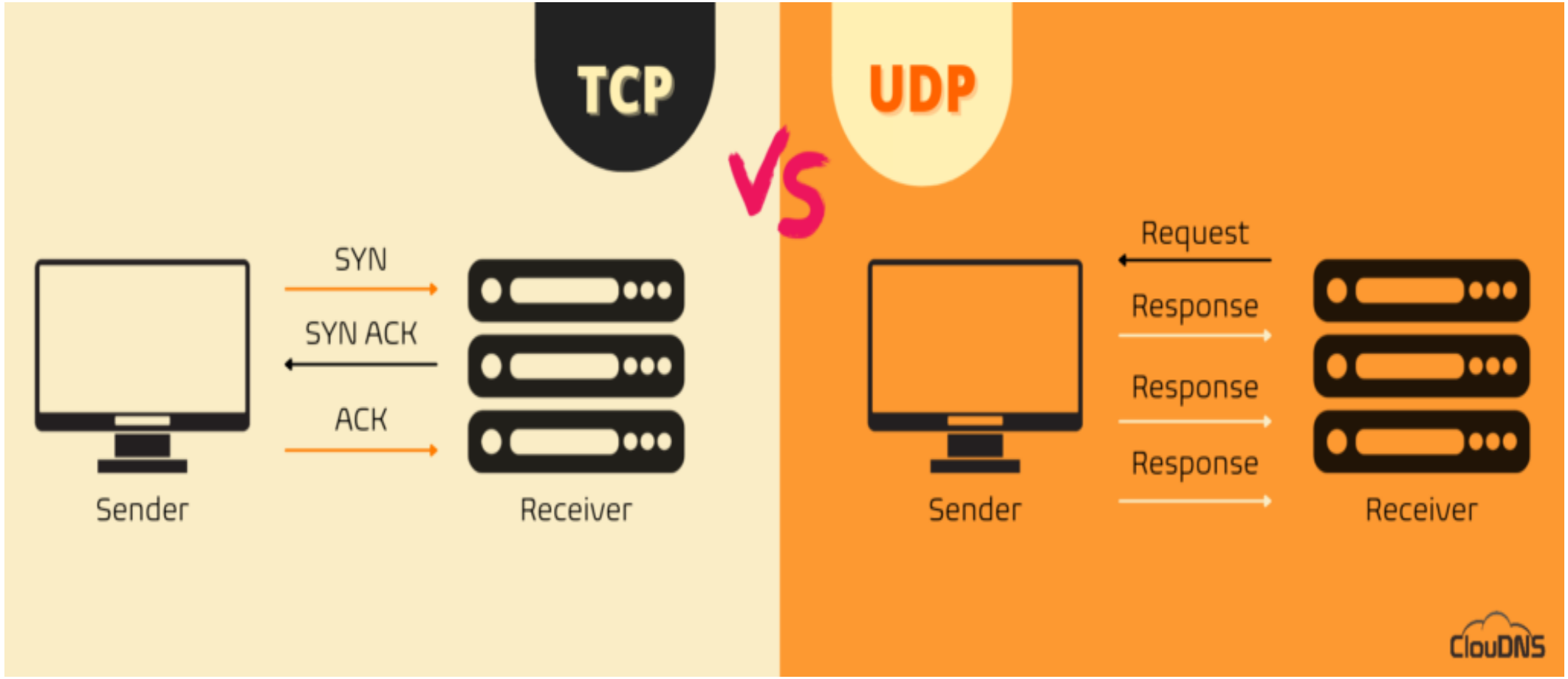
User Datagram Protocol (UDP)

- **User Datagram Protocol (UDP)** is a Transport Layer protocol.
- Unlike TCP, it is an **unreliable and connectionless protocol.**
- So, there is no need to establish a connection before data transfer.
- The UDP helps to establish low-latency and loss-tolerating connections over the network. The UDP enables process-to-process communication.



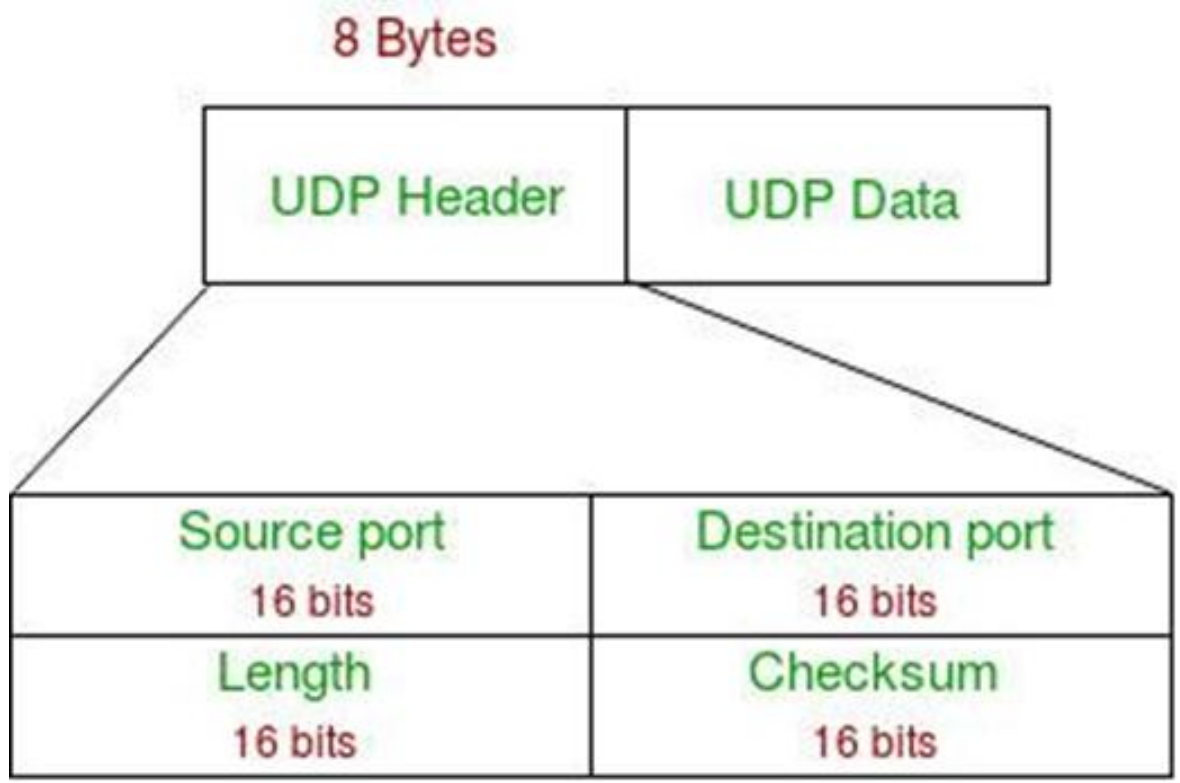
How does it work?

- UDP (User Datagram Protocol) acts in a simple way by transferring data between two devices in a network.
- It transmits packets (datagrams) straight to the target device without setting a connection, specifying the packets' order, or examining if they are delivered as arranged.
- TCP communication involves a process known as a "handshake," which establishes the connection. In addition, only when it is completed the transfer of data packets could happen.
- On the other hand, the UDP connection is not including this "handshake" process which means one device simply starts sending the information to the receiving one.
- Additionally, UDP communications do not include details about the order or confirmation for the arrival of the data. It is exactly the opposite when it comes to TCP.
- Based on these characteristics, UDP has the ability to transfer data packets a lot faster than TCP.
- The downside of a UDP connection is the loss of packets during the transit, which is not going to be resent as they are in TCP connections. Therefore, when applications implement UDP, they should be able to tolerate losses, duplications, or errors.


TCP
VS
UDP
SYN
SYN ACK
ACK
Sender
Receiver
Request
Response
Response
Response
Sender
Receiver
ClouDNS

UDP Header

- UDP header is an **8-byte** fixed and simple header, while for TCP it may vary from 20 bytes to 60 bytes.
- The first 8 Bytes contain all necessary header information and the remaining part consists of data.
- UDP port number fields are each 16 bits long, therefore the range for port numbers is defined from 0 to 65535; port number 0 is reserved.
- Port numbers help to distinguish different user requests or processes


8 Bytes
UDP Header
UDP Data
Source port
16 bits
Destination port
16 bits
Length
16 bits
Checksum
16 bits

Continued…

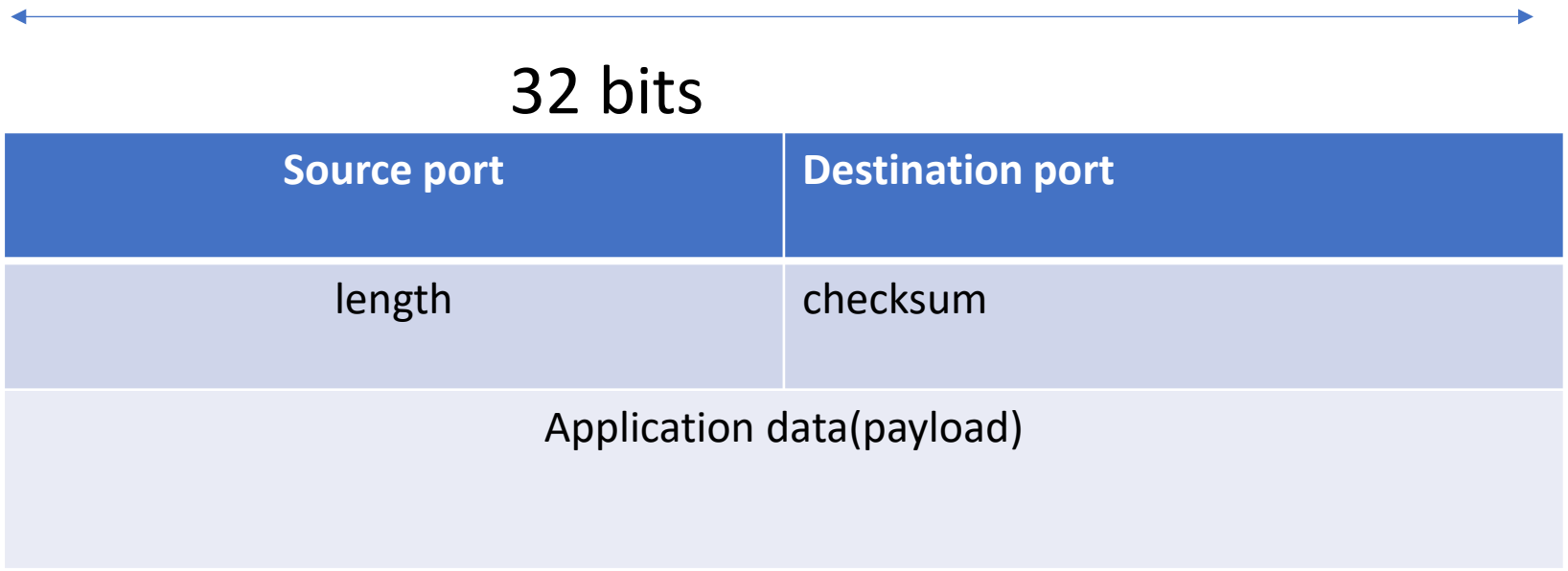
- **Source Port:** Source Port is a 2 Byte long field used to identify the port number of the source.
- **Destination Port:** It is a 2 Byte long field, used to identify the port of the destined packet.
- **Length:** Length is the length of UDP including the header and the data. It is a 16-bits field.
- **Checksum:** Checksum is 2 Bytes long field. It is the 16-bit one's complement of the one's complement sum of the UDP header, the pseudo-header of information from the IP header, and the data, padded with zero octets at the end (if necessary) to make a multiple of two octets.

Unlike TCP, the Checksum calculation is not mandatory in UDP. No Error control or flow control is provided by UDP. Hence UDP depends on IP and ICMP for error reporting. Also UDP provides port numbers so that is can differentiate between users requests.

Applications of UDP

- Used for simple request-response communication when the size of data is less and hence there is lesser concern about flow and error control.
- It is a suitable protocol for multicasting as UDP supports packet switching.
- UDP is used for some routing update protocols like RIP(Routing Information protocol).
- VOIP(Voice Over Internet Protocol) services, such as Skype and WhatsApp, use UDP for real-time voice communication. The delay in voice communication can be noticeable if packets are delayed due to congestion control, so UDP is used to ensure fast and efficient data transmission.
- DNS(Domain Name System) also uses UDP for its query/response messages. DNS queries are typically small and require a quick response time, making UDP a suitable protocol for this application.
- DHCP(Dynamic Host Configuration Protocol) uses UDP to dynamically assign IP addresses to devices on a network. DHCP messages are typically small, and the delay caused by packet loss or retransmission is generally not critical for this application.

UDP Segment -Header



UDP segmented header

UDP Checksum

Checksum Calculation:

- UDP Checksum calculation is similar to TCP Checksum computation. It’s also a 16-bit field of one’s complement of one’s complement sum of a pseudo UDP header + UDP datagram.

Sender side:

- It treats segment contents as sequence of 16-bit integers.
- All segments are added. Let's call it sum.
- Checksum: 1's complement of sum.(In 1's complement all 0s are converted into 1s and all 1s are converted into 0s).
- Sender puts this checksum value in UDP checksum field.

Receiver side:

- Calculate checksum
- All segments are added and then sum is added with sender's checksum.
- Check that any 0 bit is presented in checksum. If receiver side checksum contains any 0 then error is detected. So the packet is discarded by receiver.

Concept of Reliable Data Transfer

- Reliable Data Transfer (RDT) in the transport layer is a fundamental concept in computer networking, ensuring that data is transmitted accurately and in order across potentially unreliable networks.
- This is crucial because the underlying network layers, like the Internet Protocol (IP), do not guarantee reliable delivery.

Key Concepts of Reliable Data Transfer

1. Error Detection and Correction: To identify and correct errors that may occur during transmission, protocols use mechanisms like checksums. If a packet is found to be corrupted, it can be retransmitted to ensure data integrity.

2. Acknowledgments (ACKs) and Negative Acknowledgments (NAKs): The receiver sends an ACK to confirm successful receipt of a packet or a NAK to indicate an error, prompting the sender to retransmit the affected packet.

3. Sequence Numbers: Assigning sequence numbers to packets helps in maintaining the correct order of data and detecting duplicates. This ensures that the data is reassembled correctly at the destination

4. **Retransmission Strategies**: If an acknowledgment is not received within a certain timeframe, the sender assumes the packet was lost and retransmits it. This is managed through timers and retransmission protocols.

5. **Flow Control**: To prevent overwhelming the receiver, flow control mechanisms regulate the rate of data transmission based on the receiver's capacity to process incoming data.

6. **Congestion Control**: Protocols like TCP implement congestion control to adjust the rate of data transmission based on network traffic conditions, preventing packet loss due to congestion

Protocols Implementing Reliable Data Transfer

- **Transmission Control Protocol (TCP)**: TCP is a widely used transport layer protocol that provides reliable, ordered, and error-checked delivery of data between applications. It incorporates all the above mechanisms to ensure data integrity and efficient transmission.

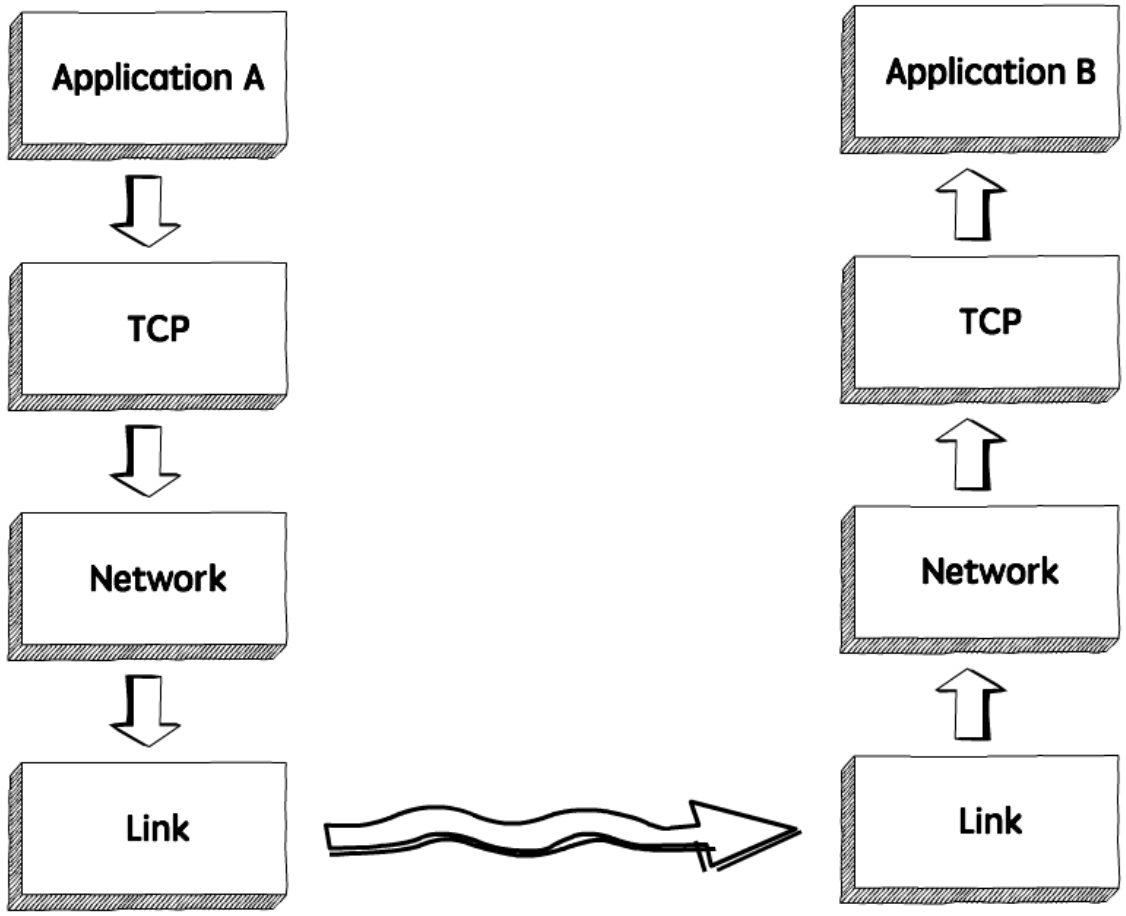
- **Automatic Repeat reQuest (ARQ)**: ARQ protocols, such as Stop-and-Wait, Go-Back-N, and Selective Repeat, are used to ensure reliable data transmission by retransmitting lost or corrupted packets based on acknowledgments received.

TCP Flow Control

- TCP is the protocol that guarantees we can have a reliable communication channel over an unreliable network.
- When we send data from a node to another, packets can be lost, they can arrive out of order, the network can be congested or the receiver node can be overloaded.
- So, the important service that TCP provides is what is called *Flow Control*.
- Flow Control basically means that TCP will ensure that a sender is not overwhelming a receiver by sending packets faster than it can consume.

How it works

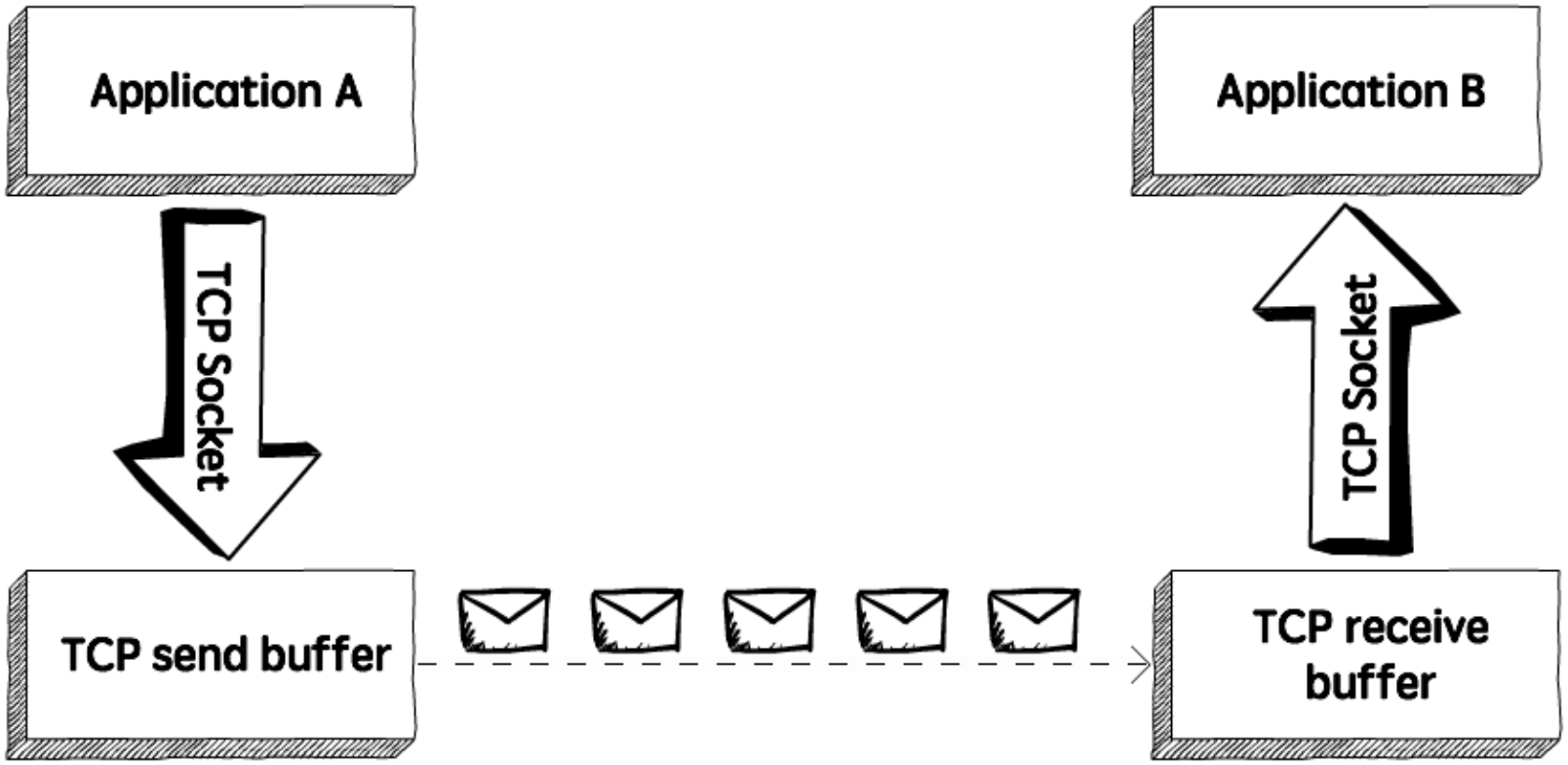
When we need to send data over a network, this is normally what happens.



Continued…

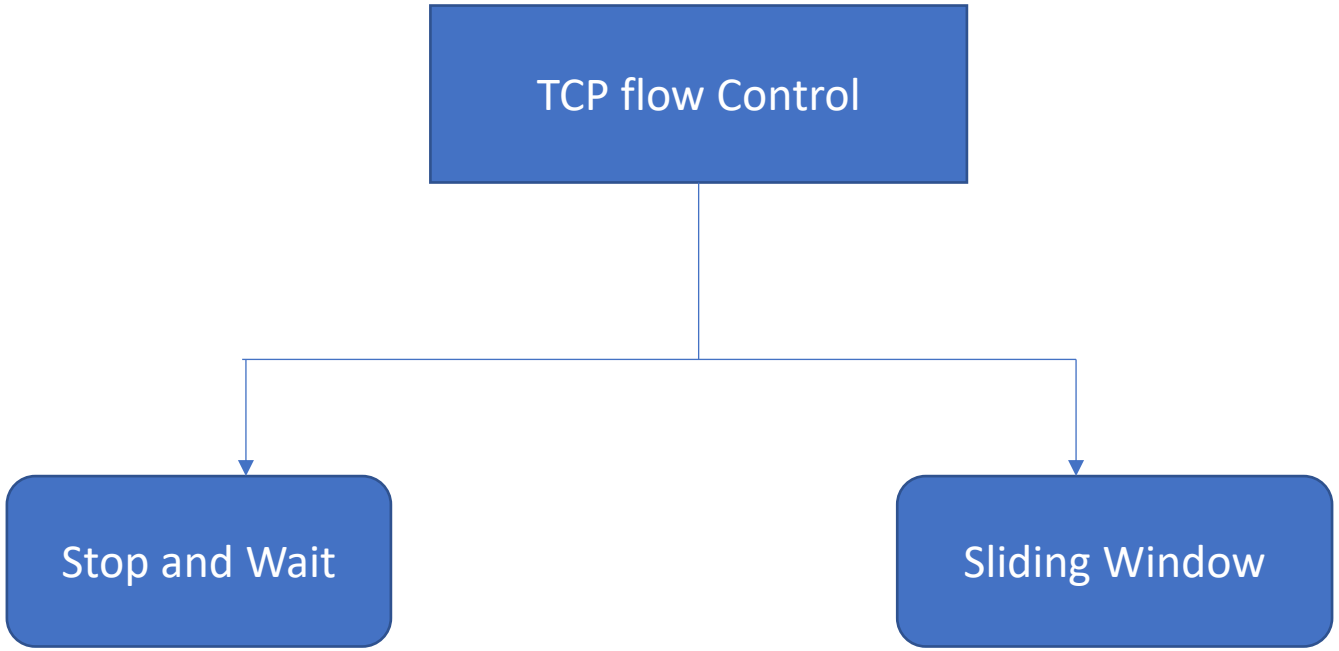
- The sender application writes data to a socket, the transport layer (in our case, TCP) will wrap this data in a segment and hand it to the network layer (e.g. IP), that will somehow route this packet to the receiving node.
- On the other side of this communication, the network layer will deliver this piece of data to TCP, that will make it available to the receiver application as an exact copy of the data sent, meaning if will not deliver packets out of order, and will wait for a retransmission in case it notices a gap in the byte stream.

If we zoom in, we will see something like this.



Continued…

- TCP stores the data it needs to send in the *send buffer*, and the data it receives in the *receive buffer*. When the application is ready, it will then read data from the receive buffer.
- Flow Control is all about making sure we don’t send more packets when the receive buffer is already full, as the receiver wouldn’t be able to handle them and would need to drop these packets.
- To control the amount of data that TCP can send, the receiver will advertise its *Receive Window (rwnd)*, that is, the spare room in the receive buffer.
- Every time TCP receives a packet, it needs to send an ack message to the sender, acknowledging it received that packet correctly, and with this ack message it sends the value of the current receive window, so the sender knows if it can keep sending data.


TCP flow Control
Stop and Wait
Sliding Window

Sliding Window Protocol - Go Back N (GBN)

- The **Sliding Window Protocol** is a method used in computer networks to manage the flow of data between two devices, ensuring that data is sent and received in the correct order.
- It works by allowing the sender to transmit multiple packets before needing an acknowledgment for each one which helps to keep the communication channel busy and improves overall data throughput.
- There are two types of sliding window protocol :

- **Go-Back-N (GBN)**
- Selective Repeat (SR)

Continued…

- In **Go-Back-N**, the sender can send multiple data packets without waiting for an acknowledgement for each one.
- However, it can only send a certain number of packets (this is called the "window size").
- If one packet is lost or not acknowledged, the sender must go back and resend that packet and all the packets that followed it, even if they were received correctly.
- While **Selective Repeat** only retransmits the specific packets that were lost or corrupted.

Go-Back-N (GBN) Protocol

- The Go-Back-N (GBN) protocol is a sliding window protocol used in networking for reliable data transmission.
- It is part of the Automatic Repeat reQuest (ARQ) protocols, which ensure that data is correctly received and that any lost or corrupted packets are retransmitted.
- For example, if packets 1, 2, 3, 4, and 5 are sent and packet 3 gets lost, the sender will have to resend packets 3, 4, and 5, even if 4 and 5 were received.

The three main characteristic features of GBN are:

1. **Sender Window Size (W_R)**

- It is N itself. If we say the protocol is GB10, then Ws = 10. N should be always greater than 1 in order to implement pipelining. For N = 1, it reduces to the stop and wait protocol.

Efficiency Of GBN = N/(1+2a)

where,

a = Tp/Tt

Tp = Propagation Delay

Tt = Transmission Delay of sender

2. Receiver Window Size (W_R)

- In the GB-N the receiver window size is one always. i.e. W_R is always 1 in GBN.

Continued…

3. Acknowledgements

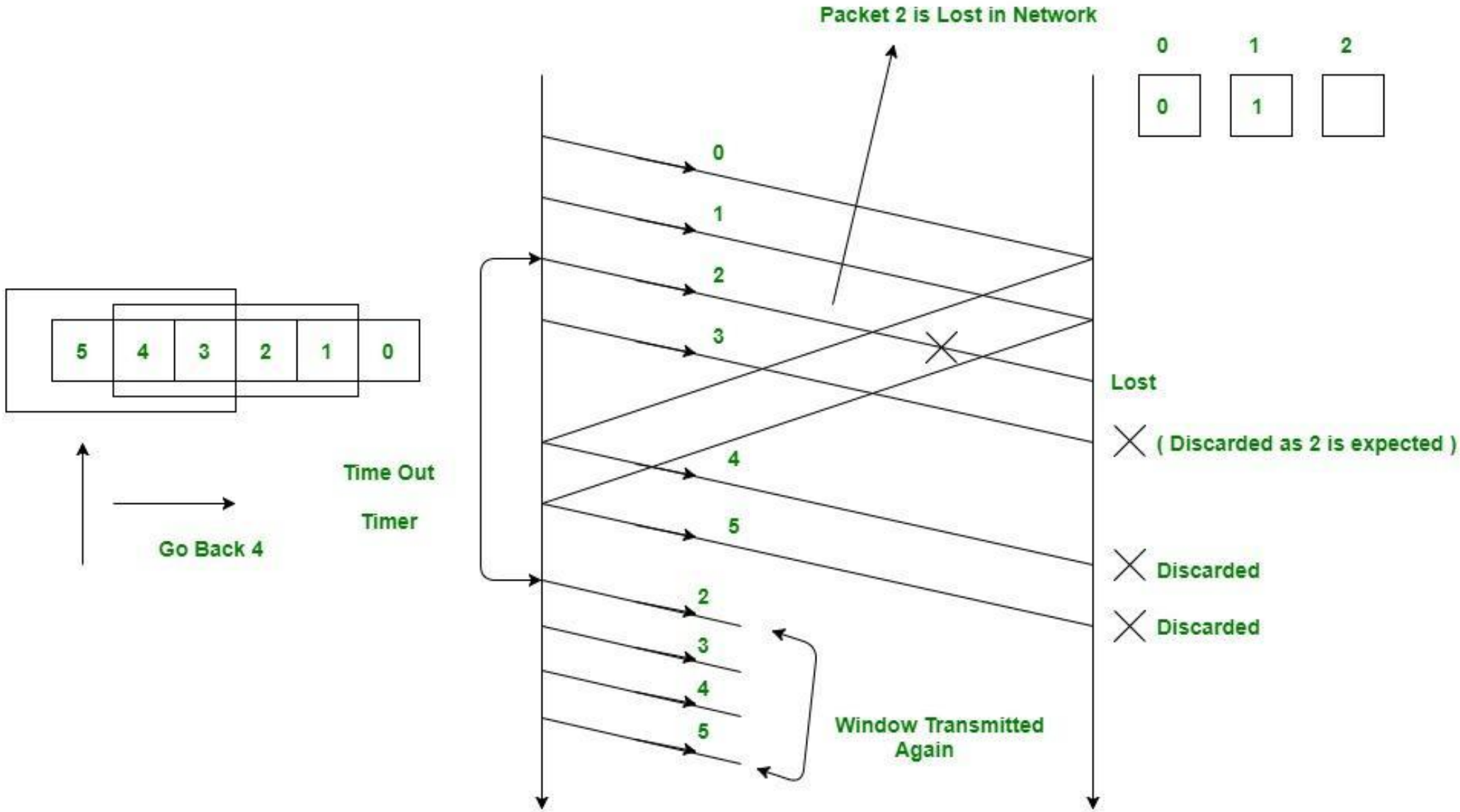
- In flow control, acknowledgments (ACKs) are signals sent by the receiver to the sender to confirm that data packets have been successfully received.
- When the sender transmits data, it waits for an acknowledgment before sending more.
- This process helps ensure that data is received correctly.
- If an acknowledgment isn't received within a certain time, the sender assumes the packet was lost and retransmits it.
- There are 2 kinds of acknowledgements namely:

➢ **Cumulative Ack**: One acknowledgement is used for many packets. The main advantage is traffic is less. A disadvantage is less reliability as if one ack is the loss that would mean that all the packets sent are lost.

➢ **Independent Ack**: If every packet is going to get acknowledgement independently then the reliability is high here but a disadvantage is that traffic is also high since for every packet we are receiving independent ack.

Working of GB-N Protocol

- Now what exactly happens in GBN, we will explain with a help of example.
- We have sender window size of 4.
- Assume that we have lots of sequence numbers just for the sake of explanation.
- Now the sender has sent the packets 0, 1, 2 and 3.
- After acknowledging the packets 0 and 1, receiver is now expecting packet 2 and sender window has also slided to further transmit the packets 4 and 5.
- Now suppose the packet 2 is lost in the network, Receiver will discard all the packets which sender has transmitted after packet 2 as it is expecting sequence number of 2.
- On the sender side for every packet send there is a time out timer which will expire for packet number 2.
- Now from the last transmitted packet 5 sender will go back to the packet number 2 in the current window and transmit all the packets till packet number 5.
- That’s why it is called Go Back N.
- Go back means sender has to go back N places from the last transmitted packet in the unacknowledged window and not from the point where the packet is lost.


Packet 2 is Lost in Network
0
1
2
0
1
5
4
3
2
1
0
Time Out
Timer
Go Back 4
0
1
2
3
Lost
(Discarded as 2 is expected)
4
5
Discarded
Discarded
2
3
4
5
Window Transmitted
Again

Relationship Between Window Size and Sequence Numbers

- The **window size** and **sequence numbers** in a sliding window protocol, like Go-Back-N or Selective Repeat, are closely related.
- The **window size** determines how many packets the sender can transmit without needing an acknowledgment. It’s like a limit on how much data can be sent before the sender has to stop and wait for confirmation.
- **Sequence numbers** are used to label packets so the receiver knows their order and can detect any missing packets.
- The **window size** should be smaller than or equal to the range of available **sequence numbers**. If the window size is too large compared to the sequence number range, the receiver might get confused because the same sequence number could be reused before the first one is acknowledged.

Continued...

- Relation between window size and sequence number is given by the formula:

$W_s + W_R <= ASN$

where,

W_s is sender window size and W_R is receiver window size, and ASN is available sequence number.

$W_s + 1 <= ASN$ because $W_R = 1$ in GB-N protocol

- **So minimum sequence numbers required in GBN = N + 1**

Bits Required in GBN = $[\log_2 (N + 1)]$

Advantages of GBN Protocol

- Simple to implement and effective for reliable communication.
- Better performance than stop-and-wait protocols for error-free or low-error networks.

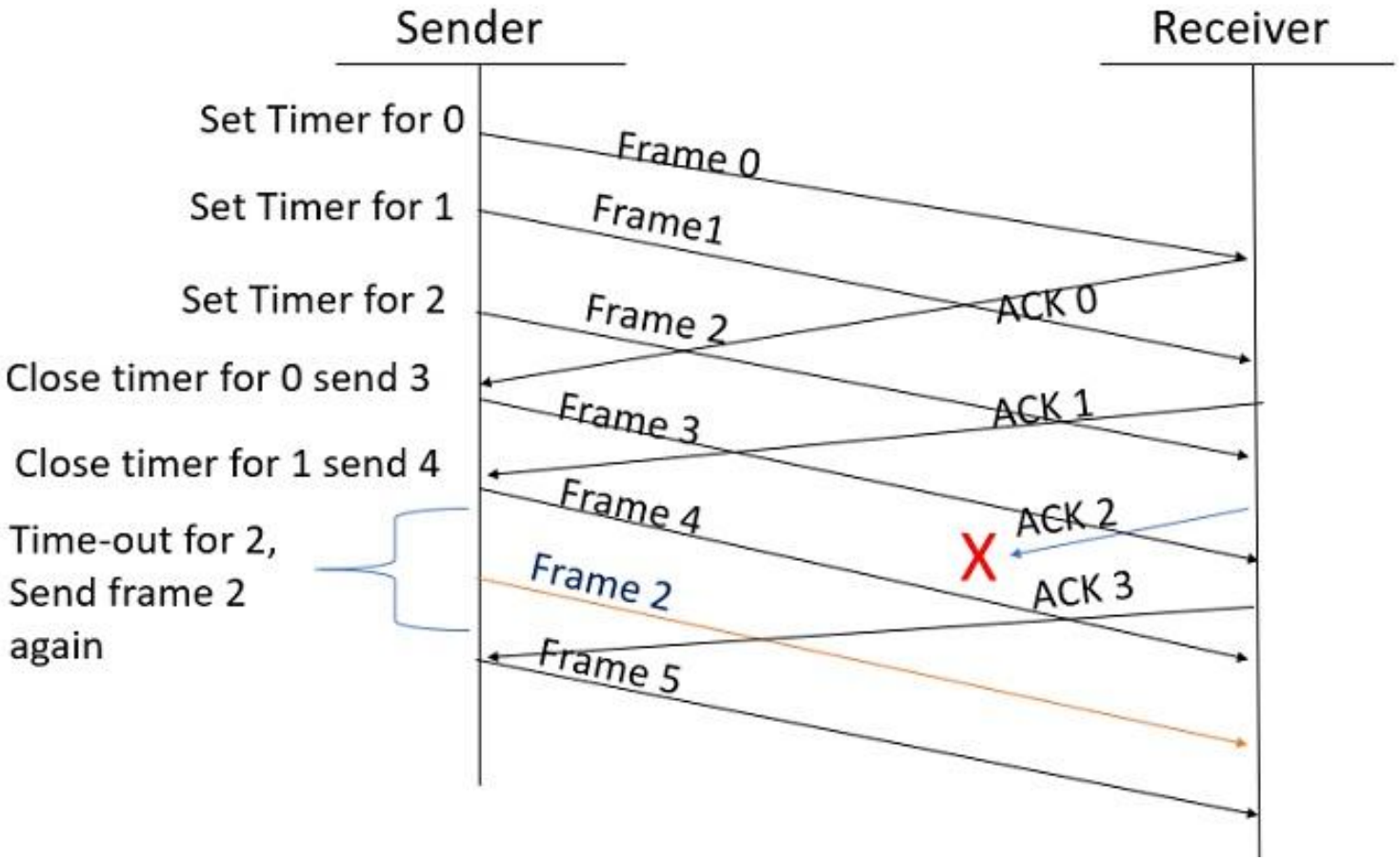
Disadvantages of GBN Protocol

- Inefficient if errors are frequent, as multiple frames might need to be retransmitted unnecessarily.
- Bandwidth can be wasted due to redundant retransmissions.

Selective Repeat Protocol (SRP)

- It is also known as Sliding Window Protocol and used for error detection and control in the data link layer.
- In the selective repeat, the sender sends several frames specified by a window size even without the need to wait for individual acknowledgement from the receiver as in Go-Back-N ARQ. In selective repeat protocol, the retransmitted frame is received out of sequence.
- In Selective Repeat ARQ only the lost or error frames are retransmitted, whereas correct frames are received and buffered.
- The receiver while keeping track of sequence numbers buffers the frames in memory and sends NACK for only frames which are missing or damaged.
- The sender will send/retransmit a packet for which NACK is received

Given below is an example of the Selective Repeat ARQ



Explanation

Step 1 Frame 0 sends from sender to receiver and set timer.

Step 2 Without waiting for acknowledgement from the receiver another frame, Frame1 is sent by sender by setting the timer for it.

Step 3 In the same way frame2 is also sent to the receiver by setting the timer without waiting for previous acknowledgement.

Step 4 Whenever sender receives the ACK0 from receiver, within the frame 0 timer then it is closed and sent to the next frame, frame 3.

Step 5 whenever the sender receives the ACK1 from the receiver, within the frame 1 timer then it is closed and sent to the next frame, frame 4.

Step 6 If the sender doesnt receive the ACK2 from the receiver within the time slot, it declares timeout for frame 2 and resends the frame 2 again, because it thought the frame2 may be lost or damaged.

Difference Between Go-Back-N and Selective Repeat Protocol

Go-Back-N Protocol	Selective Repeat Protocol
In **Go-Back N protocol**, if the sent frame are find suspected then all the frames are re-transmitted from the lost packet to the last packet transmitted.	In **Selective Repeat protocol**, only those frames are re-transmitted which are found suspected.
Sender window size of Go-Back-N Protocol is N.	Sender window size of selective Repeat protocol is also N.
Receiver window size of Go-Back-N Protocol is 1.	Receiver window size of selective Repeat protocol is N.
Go-Back-N Protocol is less complex.	Selective Repeat protocol is more complex.
In Go-Back-N Protocol, neither sender nor at receiver need sorting.	In selective Repeat protocol, receiver side needs sorting to sort the frames.
In Go-Back N protocol, type of Acknowledgement is cumulative.	In Selective Repeat protocol, type of Acknowledgement is individual.
In Go-Back-N Protocol, Out-of-Order packets are NOT Accepted (discarded) and the entire window is re-transmitted.	In selective Repeat protocol, Out-of-Order packets are Accepted.
In Go-Back-N Protocol, if Receives a corrupt packet, then also, the entire window is re-transmitted.	In selective Repeat protocol, if Receives a corrupt packet, it immediately sends a negative acknowledgement and hence only the selective packet is retransmitted.

Stop and Wait ARQ

- Stop and Wait ARQ is a Sliding Window Protocol method used for the reliable delivery of data frames.
- The stop-and-wait ARQ is used for noisy channels or links to handle flow and error control between sender and receiver.
- The Stop and Wait ARQ protocol sends a data frame and then waits for an acknowledgment (ACK) from the receiver.

What is Stop and Wait ARQ?

- The Stop and Wait ARQ protocol sends a data frame and then waits for an acknowledgment (ACK) from the receiver.
- The ACK indicates that the receiver successfully received the data frame.
- After receiving the ACK from the receiver, the sender delivers the next data frame.
- So there is a stop before the next data frame is transferred, hence it is known as the Stop and Wait ARQ protocol.

Terms in Stop and Wait Protocol

1. **Propagation Delay:** Amount of time taken by a packet to make a physical journey from one router to another router.
Propagation Delay = (Distance between routers) / (Velocity of propagation)

2. **Round Trip Time(RTT**)= Amount of time taken by a packet to reach the receiver + Time taken by the Acknowledgement to reach the sender.

3. **TimeOut (TO)** = 2* RTT

4. **Time To Live(TTL)** = 2* TimeOut. (Maximum TTL is 255 seconds)

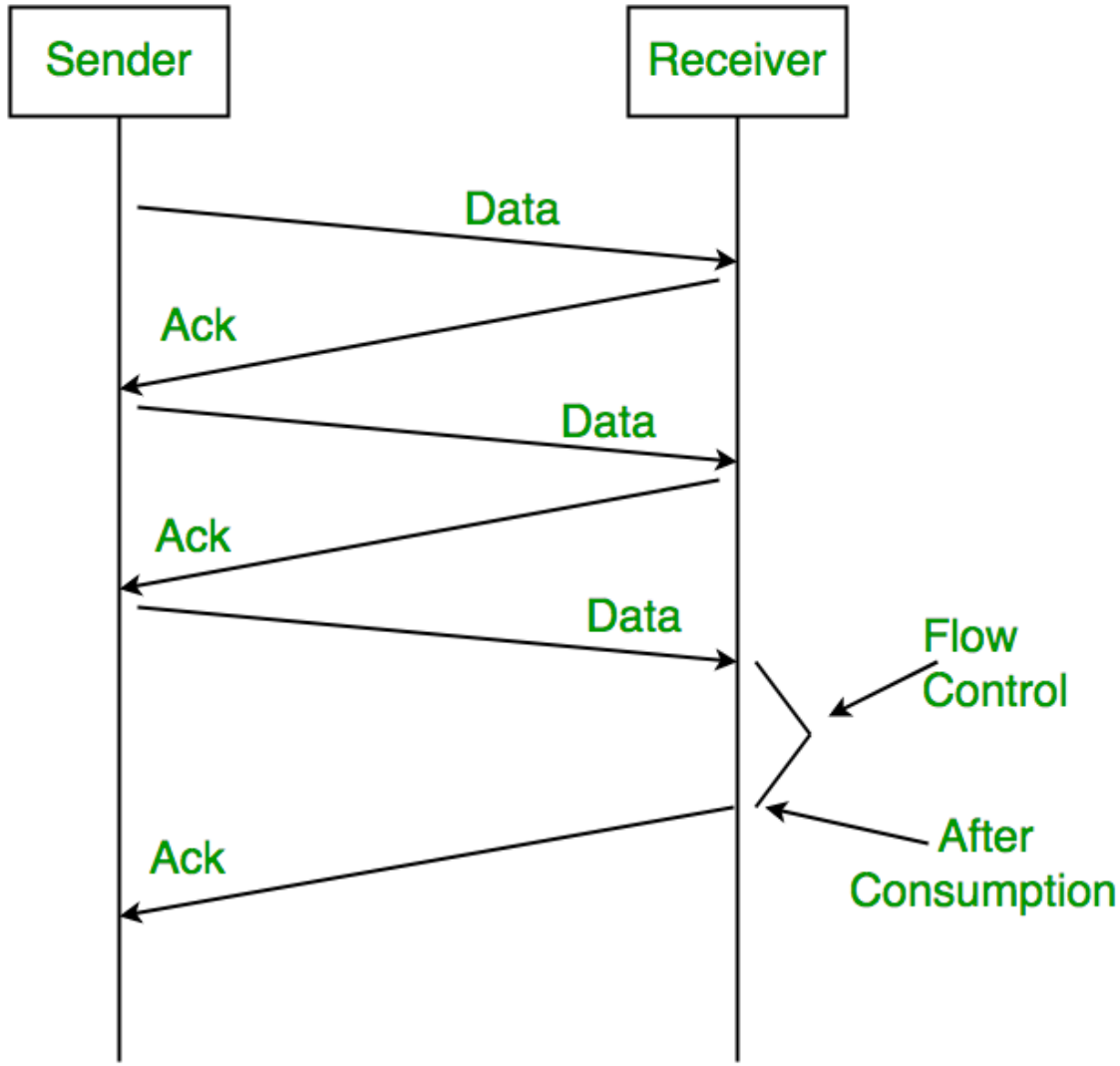
Simple Stop and Wait

At Sender

- Rule 1: Send one data packet at a time.
- Rule 2: Send the next packet only after receiving acknowledgment for the previous.

At Receiver

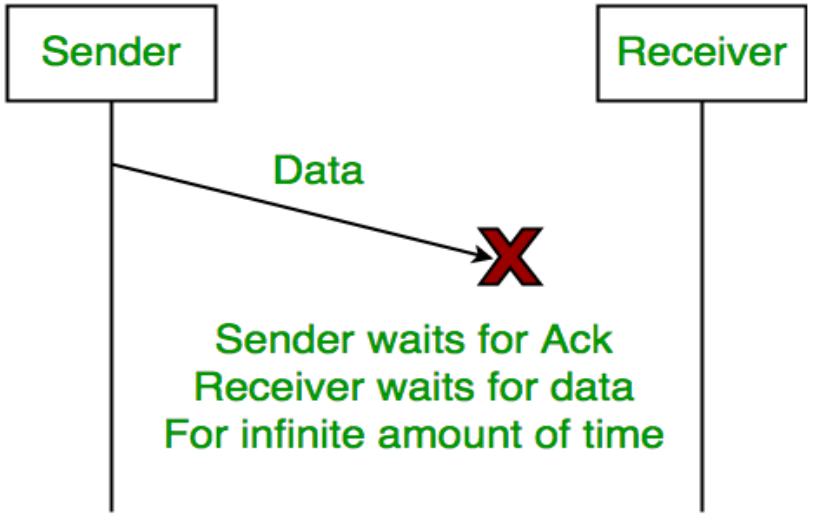
- Rule 1: Send acknowledgement after receiving and consuming a data packet.
- Rule 2: After consuming packet acknowledgement need to be sent .


Sender
Receiver
Data
Ack
Data
Ack
Data
Flow
Control
After
Consumption
Ack

Problems Associated with Stop and Wait

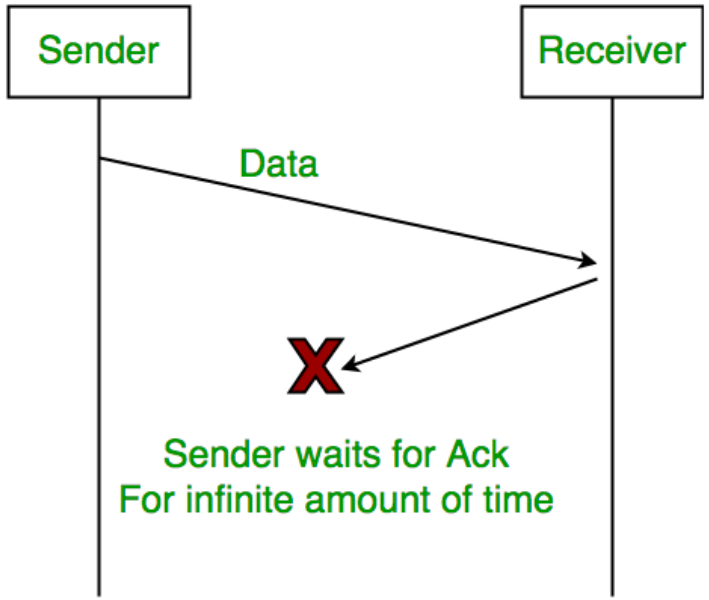
1. Lost Data

Assume the sender transmits the data packet and it is lost. The receiver has been waiting for the data for a long time. Because the data is not received by the receiver, it does not transmit an acknowledgment. The sender does not receive an acknowledgment, it will not send the next packet. This problem is caused by a loss of data.



2. Lost Acknowledgement

Assume the sender sends the data, which is also received by the receiver. The receiver sends an acknowledgment after receiving the packet. In this situation, the acknowledgment is lost in the network. The sender does not send the next data packet because it does not receive acknowledgement, under the stop and wait protocol, the next packet cannot be transmitted until the preceding packet's acknowledgment is received.



3. Delayed Acknowledgement/Data

- Assume the sender sends the data, which is also received by the receiver. The receiver then transmits the acknowledgment, which is received after the sender's timeout period. After a timeout on the sender side, a long-delayed acknowledgement might be wrongly considered as acknowledgement of some other recent packet.

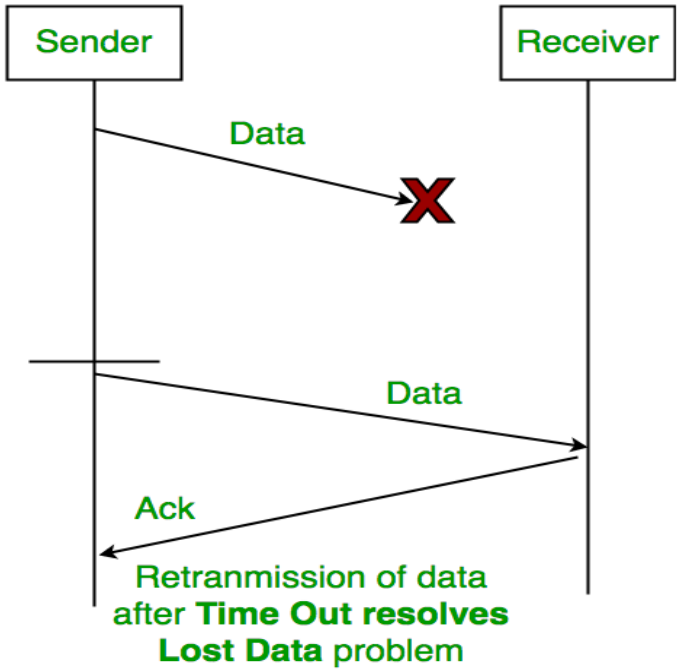
Stop and Wait for ARQ (Automatic Repeat Request)

- The above 3 problems are resolved by Stop and Wait for ARQ (Automatic Repeat Request) that does both error control and flow control.



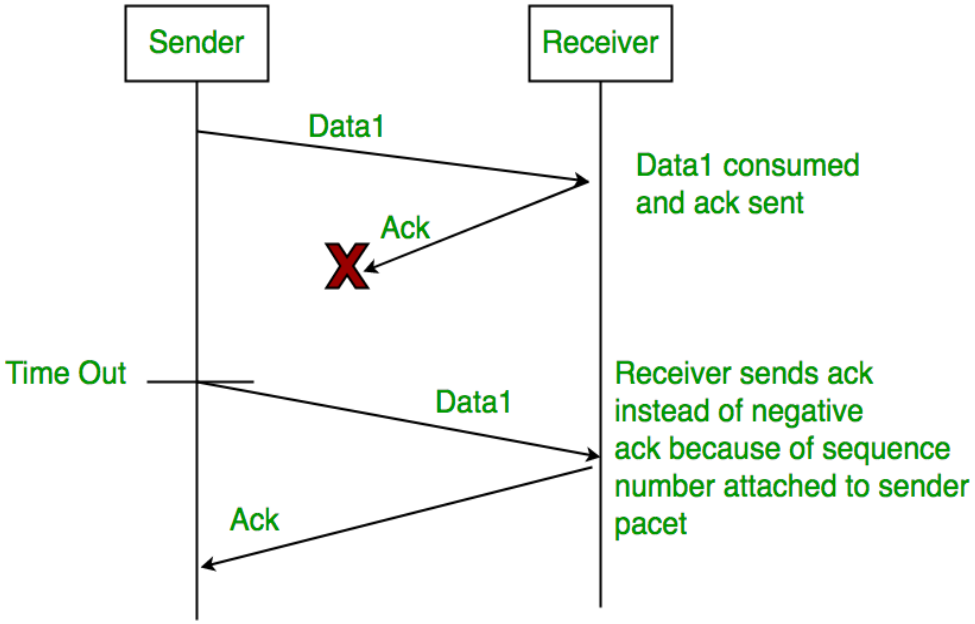
1. Time Out

- Timeout refers to the duration for which the sender waits for an acknowledgment (ACK) from the receiver after transmitting a data packet. If the sender does not receive an ACK within this timeout period, it assumes that the frame was lost or corrupted and retransmits the frame.



2. Sequence Number (Data)

- In Stop-and-Wait ARQ, the sender assigns sequence numbers to each data frame it sends. This allows the receiver to identify and acknowledge each frame individually, ensuring reliable delivery of data packets. After sending a frame, the sender waits for an acknowledgment before sending the next frame.
-

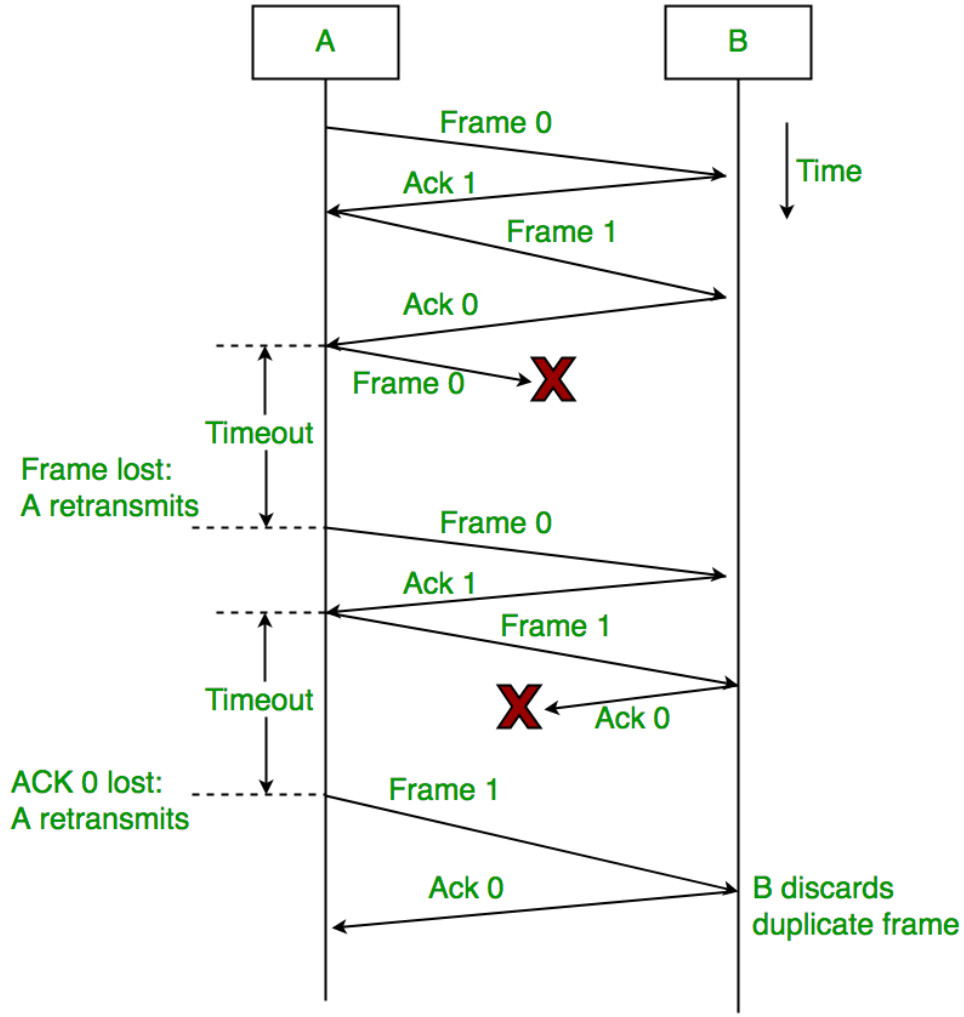


3. Sequence Number(Acknowledgement)

- Similarly, sequence numbers are also used in acknowledgments (ACKs) sent by the receiver to acknowledge received data frames. When the receiver successfully receives a data frame, it sends an ACK back to the sender, indicating the sequence number of the next expected frame. The sender uses this ACK to determine whether the transmission was successful and whether it can proceed to send the next frame.

Working of Stop and Wait for ARQ

- Sender A sends a data frame or packet with sequence number 0.
- Receiver B, after receiving the data frame, sends an acknowledgement with sequence number 1 (the sequence number of the next expected data frame or packet)
  There is only a one-bit sequence number that implies that both sender and receiver have a buffer for one frame or packet only.
- It uses a 1-bit sequence number that alternates 0→1→0… so the receiver can detect duplicates and the sender can recover from loss using **timeouts + retransmissions**.


A
B
Frame 0
Time
Ack 1
Frame 1
Ack 0
Frame 0
Timeout
Frame lost:
A retransmits
Frame 0
Ack 1
Frame 1
Timeout
Ack 0
ACK 0 lost:
A retransmits
Frame 1
Ack 0
B discards
duplicate frame

Advantages of Stop and Wait ARQ

- **Simple Implementation:** Stop and Wait ARQ is a simple protocol that is easy to implement in both hardware and software. It does not require complex algorithms or hardware components, making it an inexpensive and efficient option.
- **Error Detection:** Stop and Wait ARQ detects errors in the transmitted data by using checksums or cyclic redundancy checks(CRC). If an error is detected, the receiver sends a negative acknowledgment (NAK) to the sender, indicating that the data needs to be retransmitted.
- **Reliable:** Stop and Wait ARQ ensures that the data is transmitted reliably and in order. The receiver cannot move on to the next data packet until it receives the current one. This ensures that the data is received in the correct order and eliminates the possibility of data corruption.
- **Flow Control:** Stop and Wait ARQ can be used for flow control, where the receiver can control the rate at which the sender transmits data. This is useful in situations where the receiver has limited buffer space or processing power.
- **Backward Compatibility:** Stop and Wait ARQ is compatible with many existing systems and protocols, making it a popular choice for communication over unreliable channels.

Disadvantages of Stop and Wait ARQ

- **Low Efficiency:** Stop and Wait ARQ has low efficiency as it requires the sender to wait for an acknowledgment from the receiver before sending the next data packet. This results in a low data transmission rate, especially for large data sets.
- **High Latency:** Stop and Wait ARQ introduces additional latency in the transmission of data, as the sender must wait for an acknowledgment before sending the next packet. This can be a problem for real-time applications such as video streaming or online gaming.
- **Limited Bandwidth Utilization:** Stop and Wait ARQ does not utilize the available bandwidth efficiently, as the sender can transmit only one data packet at a time. This results in underutilization of the channel, which can be a problem in situations where the available bandwidth is limited.
- **Limited Error Recovery:** Stop and Wait ARQ has limited error recovery capabilities. If a data packet is lost or corrupted, the sender must retransmit the entire packet, which can be time-consuming and can result in further delays.
- **Vulnerable to Channel Noise:** Stop and Wait ARQ is vulnerable to channel noise, which can cause errors in the transmitted data. This can result in frequent retransmissions and can impact the overall efficiency of the protocol.

TCP Congestion Control

- TCP congestion control is a method used by the TCP protocol to manage data flow over a network and prevent congestion.
- TCP uses a congestion window and congestion policy that avoids congestion.
- Previously, we assumed that only the receiver could dictate the sender's window size.
- We ignored another entity here, the network.
- If the network cannot deliver the data as fast as it is created by the sender, it must tell the sender to slow down.
- In other words, in addition to the receiver, the network is a second entity that determines the size of the sender's window.

Congestion Policy in TCP

Slow Start Phase: Starts slow increment is exponential to the threshold.

Congestion Avoidance Phase: After reaching the threshold increment is by 1.

Congestion Detection Phase: The sender goes back to the Slow start phase or the Congestion avoidance phase.

Slow Start Phase

Exponential Increment: In this phase after every RTT the congestion window size increments exponentially.

the **congestion window (cwnd) controls how much data the sender can send before waiting for acknowledgements (ACKs).

Example: If the initial congestion window size is 1 segment, and the first segment is successfully acknowledged, the congestion window size becomes 2 segments. If the next transmission is also acknowledged, the congestion window size doubles to 4 segments. This exponential growth continues as long as all segments are successfully acknowledged.

Initial congestion window = 1 segment

RTT	Congestion Window	What happens?
Start	1 segment	Send 1 segment
After 1 RTT	2 segments	ACK received → window increases
After 2 RTTs	4 segments	ACKs received → window doubles
After 3 RTTs	8 segments	ACKs received → window doubles
After 4 RTTs	16 segments	ACKs received → window doubles

Congestion Avoidance Phase

Additive Increment: This phase starts after the threshold value also denoted as **ssthresh.** The size of CWND (Congestion Window) increases additive. After each RTT cwnd = cwnd + 1.

For example: if the congestion window size is 20 segments and all 20 segments are successfully acknowledged within an RTT, the congestion window size would be increased to 21 segments in the next RTT. If all 21 segments are again successfully acknowledged, the congestion window size will be increased to 22 segments, and so on.

Congestion Detection Phase

- **Multiplicative Decrement:**

➢ If congestion occurs, the congestion window size is decreased.

➢ The only way a sender can guess that congestion has happened is the need to retransmit a segment.

➢ Retransmission is needed to recover a missing packet that is assumed to have been dropped by a router due to congestion.

➢ Retransmission can occur in one of two cases: when the RTO timer times out or when three duplicate ACKs are received.